

L0 Self-Service Knowledge Base Chatbot

TEAM MEMEBERS

ASHMIKA R

BALASURYA D

ASHUTOSH JAIRAM S

BHUPESH S

1. Introduction

1.1 Project Overview

The L0 Self-Service Knowledge Base Chatbot is an AI-powered support assistant designed to automate responses to frequently asked user queries. The system leverages Retrieval-Augmented Generation (RAG) to retrieve information from an internal knowledge base and generate context-aware answers with source citations.

Unlike traditional chatbots that rely solely on large language models, this system ensures that responses are grounded in verified knowledge base articles. In addition, the chatbot computes a confidence score for each response and automatically escalates unresolved or uncertain queries to a support ticketing system.

The solution aims to reduce repetitive support workload, improve response time, and provide users with transparent and trustworthy answers.

1.2 Problem Statement

Organizations frequently receive repetitive support questions such as:

- ❖ How do I reset my password?
- ❖ How can I update billing information?
- ❖ How do I manage my account?
- ❖ How can I configure security settings?

Handling these requests manually increases operational cost and response time while reducing the productivity of support teams.

Traditional chatbots often generate answers without referencing trusted sources, which can lead to inaccurate or hallucinated responses.

1.3 Proposed Solution

The proposed system provides an intelligent self-service support platform that:

- ❖ Retrieves information from Markdown knowledge base articles.
- ❖ Uses vector similarity search to find relevant content.
- ❖ Generates answers using Gemini API.

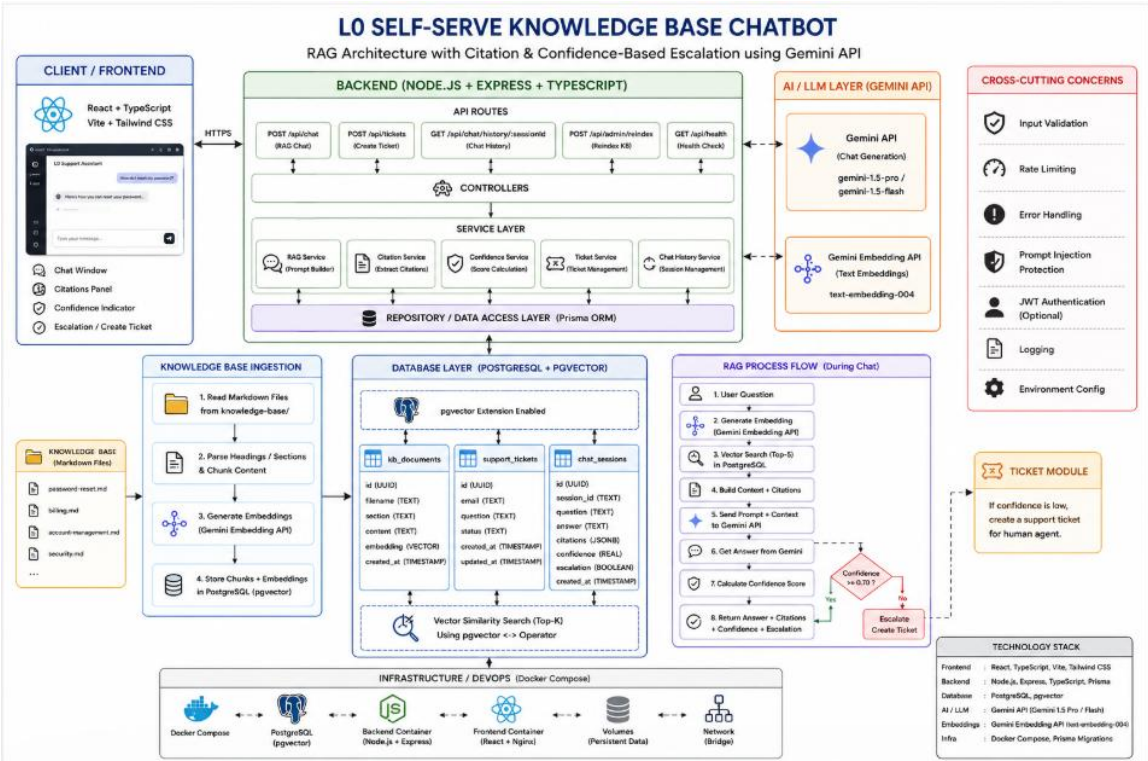
- ❖ Provides citations for transparency.
- ❖ Calculates confidence scores.
- ❖ Escalates uncertain queries by creating support tickets.

2. Objectives

The main objectives of this project are:

- ❖ Automate repetitive support queries.
- ❖ Reduce support team workload.
- ❖ Improve response accuracy.
- ❖ Provide source citations for every answer.
- ❖ Minimize hallucination by grounding responses in knowledge base content.
- ❖ Escalate low-confidence queries to human agents.
- ❖ Maintain conversation history.

3. System Architecture



4. Technology Stack

Frontend Technologies

Technology Purpose

React	User Interface Development
TypeScript	Type Safety
Vite	Build Tool
Tailwind CSS	Styling
Lucide React	Icons

Backend Technologies

Technology Purpose

Technology Purpose

Node.js Runtime Environment

Express	REST API Framework
TypeScript	Strong Typing
Prisma ORM	Database Access Layer

Database Technologies

Technology Purpose

PostgreSQL	Relational Database
pgvector	Vector Similarity Search

AI Components

Chat Generation

- ❖ Gemini 1.5 Pro
- ❖ Gemini 1.5 Flash

Embedding Model

- ❖ text-embedding-004

Infrastructure Components

- ❖ Docker
- ❖ Docker Compose
- ❖ npm Workspaces
- ❖ Prisma Migrations

5. Functional Requirements

FR1: Chat Query Processing

The system shall:

- ❖ Accept user questions.
- ❖ Generate embeddings.
- ❖ Perform vector similarity search.
- ❖ Retrieve the most relevant chunks.
- ❖ Generate answers using Gemini API.
- ❖ Return confidence score and citations.

FR2: Knowledge Base Ingestion

The system shall:

- ❖ Read Markdown files.
- ❖ Parse sections and headings.
- ❖ Chunk content into smaller segments.
- ❖ Generate embeddings.
- ❖ Store chunks in PostgreSQL.

FR3: Citation Support

The system shall provide source references in the following format:

filename > section

No answer shall be returned without citations.

FR4: Confidence Scoring

Confidence score is computed using:

- ❖ Similarity Score → 60%
- ❖ Chunk Support → 20%
- ❖ LLM Confidence → 20%

Threshold:

0.70

Queries below the threshold are escalated to the ticketing system.

FR5: Ticket Creation

The system shall:

- ❖ Create support tickets.
- ❖ Store user email.
- ❖ Store user question.
- ❖ Set status as OPEN.

FR6: Chat History

The system shall:

- ❖ Store every conversation.
- ❖ Retrieve previous interactions.
- ❖ Maintain session information.

6. Non-Functional Requirements

Performance

- ❖ Average response time below five seconds.

Scalability

- ❖ Support concurrent users.

Reliability

- ❖ Accurate retrieval mechanism.

Security

- ❖ Input validation.
- ❖ Prompt injection protection.
- ❖ Rate limiting.
- ❖ API key security.

Availability

- ❖ Continuous operation with Docker containers.

7. Module Design

Module 1: Project Setup Module

This module establishes the foundation of the application.

Backend

- ❖ Node.js 22
- ❖ Express
- ❖ TypeScript

Frontend

- ❖ React
- ❖ Vite
- ❖ Tailwind CSS

Features

- ❖ Health API
- ❖ Environment configuration
- ❖ npm scripts

Module 2: Database + Prisma Module

This module provides persistent storage.

Tables

kb_documents

- ❖ Stores:
- ❖ Document chunks
- ❖ Metadata
- ❖ Embeddings

support_tickets

Stores:

- ❖ User email
- ❖ Question
- ❖ Ticket status

chat_sessions

Stores:

- ❖ Conversation history
- ❖ Confidence score
- ❖ Escalation status

Module 3: Knowledge Base Ingestion Module

This module processes Markdown files and stores vector embeddings.

Workflow

Markdown Files



Parser



Chunking



Embedding Generation



PostgreSQL

Module 4: Docker Infrastructure Module

Responsible for containerization.

Components:

- ❖ PostgreSQL container
- ❖ Backend container
- ❖ Frontend container
- ❖ Persistent volume storage

Module 5: Vector Search Module

Responsibilities:

- ❖ Generate query embeddings.
- ❖ Perform vector similarity search.
- ❖ Retrieve top-k chunks.
- ❖ Return similarity scores.

Module 6: RAG Chat Module

Workflow:

User Question



Embedding Generation



Vector Search



Top Matching Chunks



Prompt Builder



Gemini API



Generated Answer



Confidence Calculation



Escalation Decision

Module 7: Citation Module

Responsibilities:

- ❖ Extract citations.
- ❖ Format citations.
- ❖ Ensure answer transparency.

Format:

filename > section

Module 8: Ticket Module

Responsibilities:

- ❖ Create support tickets.
- ❖ Store email and question.
- ❖ Maintain ticket status.

Default Status:

OPEN

Module 9: Chat History Module

Responsibilities:

- ❖ Store interactions.
- ❖ Retrieve session history.

- ❖ Maintain user conversations.

Module 10: Security Module

Features:

- ❖ Input validation
- ❖ Error handling
- ❖ Rate limiting
- ❖ Prompt injection protection
- ❖ JWT authentication support

Module 11: Frontend Module

Components:

- ❖ ChatWindow
- ❖ MessageList
- ❖ MessageBubble
- ❖ ChatInput
- ❖ Loading States

Module 12: Citation and Confidence Module

Components:

- ❖ CitationPanel
- ❖ ConfidenceIndicator
- ❖ Escalation Button

Module 13: Ticket Modal Module

Components:

- ❖ Email Input
- ❖ Question Preview
- ❖ Submit Button
- ❖ Success State

Module 14: Testing Module

Includes:

- ❖ Unit Tests
- ❖ API Integration Tests
- ❖ Vector Search Tests
- ❖ Confidence Calculation Tests
- ❖ Ticket Endpoint Tests

Module 15: Deployment Module

Includes:

- ❖ Dockerfile
- ❖ Docker Compose
- ❖ PostgreSQL with pgvector
- ❖ Prisma migrations